199. Binary Tree Right Side View Medium

Acceptance: 69.2% | LeetCode

Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom*.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:

Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

Claude Opus Solution

Binary Tree Right Side View

1. Intuition

The **key insight**: When you stand on the right side of a tree, at each *level* you can only see the **rightmost node** of that level — everything behind it is blocked.

So the problem reduces to: **for each level of the tree, find the last (rightmost) node's value.**

This transforms a seemingly geometric/visual problem into a standard **level-by-level traversal** (BFS). At each level, we simply grab the last node we process.

2. Approach

We use **Breadth-First Search (BFS)** with a queue, processing the tree one full level at a time:

```
1. If root is None, return empty list.
2. Initialize a queue containing the root.
3. While queue is not empty:
   a. Record the current number of nodes in the queue (= size of this level).
   b. Loop exactly that many times:
        - Pop a node from the front of the queue.
        - Enqueue its left child (if any), then right child (if any).
        - Track the LAST node processed in this loop.
   c. Append the last node's value to the result (this is the rightmost node).
4. Return the result list.
```

The trick of capturing `len(queue)` **before** the inner loop ensures we process exactly one level per outer iteration.

3. Python Solution

```python
from collections import deque
from typing import List, Optional


# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


class Solution:
```

```python
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        # Edge case: empty tree -> nothing to see
        if root is None:
            return []

        result = []
        queue = deque([root])  # BFS queue, start with the root

        while queue:
            # Number of nodes currently in this level.
            # We snapshot it BEFORE processing because we'll
            # enqueue children during the loop (next level).
            level_size = len(queue)
            rightmost_val = None

            # Process every node in the current level
            for _ in range(level_size):
                node = queue.popleft()      # take node from front
                rightmost_val = node.val    # last assignment = rightmost node

                # Enqueue children left-to-right so the
                # last one processed next level is the rightmost
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

            # After the loop, rightmost_val holds the last node
            # processed in this level = the one we can "see"
            result.append(rightmost_val)

        return result
```

Note: Compared to the reference solution, this version avoids building a `temp` list per level (saving memory). We only track the single rightmost value with `rightmost_val`.

---

4. Complexity Analysis

Metric	Complexity	Explanation
Time	`O(n)`	Each node is enqueued and dequeued exactly once.
Space	`O(n)`	In the worst case (a complete tree), the queue holds up to `n/2` nodes at the bottom level → `O(n)`. The output list holds at most one value per level.

Where `n` is the number of nodes in the tree.

---

5. Alternative Approaches

A. DFS (Right-First Pre-order Traversal)

Visit the **right subtree before the left**. The **first** node we encounter at each depth is the rightmost visible one.

```python
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        result = []

        def dfs(node, depth):
            if not node:
                return
            # If this is the first node we reach at this depth,
            # it must be the rightmost (because we go right first).
            if depth == len(result):
                result.append(node.val)
            dfs(node.right, depth + 1)  # right child first!
            dfs(node.left, depth + 1)
```

```
        dfs(root, 0)
        return result
```

- **Time:** `O(n)`, **Space:** `O(h)` where `h` is tree height (recursion stack).
- This can be more space-efficient than BFS for **tall, sparse trees** (`O(h)` vs `O(n)`).

B. DFS with Left-First Traversal

Same as above but go left first, **overwriting** the value at each depth. The last write at each depth wins (the rightmost). Slightly less elegant since it does redundant overwrites, but still valid.

Summary of Key Patterns

- **Level Order Traversal (BFS)**: The go-to template whenever a problem cares about "per level" properties.
- **Snapshot `len(queue)`**: Standard trick to isolate one level at a time.
- **Direction-aware DFS**: Visiting children in a chosen order lets you exploit "first-seen-at-depth" logic.

Codex Solution

1. Intuition

The **right side view** contains exactly **one node per depth/level**: the node that appears farthest to the right on that level.

So the key pattern is:

> Traverse the tree level by level, and for each level, record the last node visited.

This naturally suggests **Breadth-First Search BFS / Level Order Traversal**.

For example:

```
      1
     / \
    2   3
     \   \
      5   4
```

Level by level:

```
Level 0: [1]       -> rightmost = 1
Level 1: [2, 3]    -> rightmost = 3
Level 2: [5, 4]    -> rightmost = 4
```

Answer:

```
[1, 3, 4]
```

---

2. Approach

We use a queue to perform BFS.

Pseudocode

```
function rightSideView(root):
    if root is null:
        return []

    result = []
    queue = [root]

    while queue is not empty:
        level_size = size of queue

        for i from 0 to level_size - 1:
            node = queue.pop_front()

            if i == level_size - 1:
                add node.val to result

            if node.left exists:
```

```
                queue.push_back(node.left)

            if node.right exists:
                queue.push_back(node.right)

    return result
```

Why this works

At the start of each loop, the queue contains all nodes of the current level.

We process exactly `level_size` nodes. The last node processed in that group is the rightmost node of that level, so we add it to the answer.

---

3. Python Solution

```python
from collections import deque
from typing import List, Optional


# Definition for a binary tree node.
# LeetCode already provides this class.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Value stored in the node
        self.left = left        # Left child
        self.right = right      # Right child


class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        # If the tree is empty, there is nothing to see.
        if root is None:
            return []

        # This will store the visible node values from top to bottom.
        result = []

        # Queue for BFS traversal.
        # We start with the root node.
        queue = deque([root])

        # Process the tree level by level.
        while queue:
            # Number of nodes in the current level.
            level_size = len(queue)

            # Visit every node in the current level.
            for i in range(level_size):
                node = queue.popleft()

                # If this is the last node of the current level,
                # then it is the rightmost visible node.
                if i == level_size - 1:
                    result.append(node.val)

                # Add left child first.
                # This ensures nodes are processed left-to-right.
                if node.left is not None:
                    queue.append(node.left)

                # Add right child second.
                # Therefore, the last node processed at each level
                # will be the rightmost node.
                if node.right is not None:
                    queue.append(node.right)

        return result
```

---

4. Complexity Analysis

Let `n` be the number of nodes in the binary tree.

Time Complexity

```
O(n)
```

Each node is visited exactly once.

Space Complexity

```
O(n)
```

The queue can store up to one full level of the tree.

In the worst case, the widest level can contain about `n / 2` nodes, so the space complexity is `O(n)`.

The result list also stores at most one node per level, which is `O(h)`, where `h` is the height of the tree. However, the queue dominates in the worst case.

---

5. Alternative Approaches

Alternative 1: DFS, Visit Right First

Instead of BFS, we can use Depth-First Search.

The idea is:

> Visit the right child before the left child. The first node we see at each depth is the visible node from the right side.

```python
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        result = []

        def dfs(node: Optional[TreeNode], depth: int) -> None:
            # Base case: empty node
            if node is None:
                return

            # If this is the first node we have visited at this depth,
            # then it must be the rightmost node because we visit right first.
            if depth == len(result):
                result.append(node.val)

            # Visit right subtree before left subtree.
            dfs(node.right, depth + 1)
            dfs(node.left, depth + 1)

        dfs(root, 0)
        return result
```

DFS Complexity

Time complexity:

```
O(n)
```

Space complexity:

```
O(h)
```

where `h` is the height of the tree due to recursion stack.

In the worst case of a skewed tree, `h = n`, so space is `O(n)`. In a balanced tree, space is `O(log n)`.

Summary

Approach	Idea	Time	Space
BFS Level Order	Take last node of each level	`O(n)`	`O(n)`
DFS Right First	First node seen at each depth	`O(n)`	`O(h)`